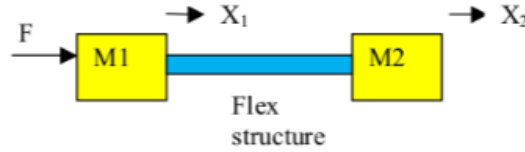


TEK4090 – Introduction to Modern Control

Assignment 2 v1.0

Due: November 17 2025, 23:30

- (40 points) Consider the following model of two masses connected by a flexible structure, with a lateral force on one of the masses.



A dynamical system can be written as,

$$(M_{lump} + M_{rod})\ddot{U} + C_{rod}\dot{U} + K_{rod}U = \begin{bmatrix} 1 \\ 0 \end{bmatrix} F, \quad (1)$$

where $U = [x_1, x_2]^T \in \mathbb{R}^2$ is the displacement of the two masses. Given the density ρ_r of the rod, the elastic modulus E_r of the rod, the cross-sectional area A_r of the rod, the length L_r of the rod, and the masses M_1 and M_2 , the system matrices can be written as,

$$M_{lump} = \begin{bmatrix} M_1 & 0 \\ 0 & M_2 \end{bmatrix}, \quad M_{rod} = \frac{A_r L_r \rho_r}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad (2)$$

$$C_{rod} = \alpha K_{rod}, \quad K_{rod} = \frac{A_r E_r}{L_r} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \quad (3)$$

For simplicity, let

$$\frac{A_r L_r \rho_r}{6} = 1; \quad \frac{A_r E_r}{L_r} = 1; \quad M_1 = M_2 = 10; \quad \alpha = 0.1. \quad (4)$$

Finally, let the input be F and the output be x_2 .

- Obtain the state space model $\Sigma := (A, B, C, D)$ of the system. Use Matlab to find the poles and zeros of the system.

Ans: We start off by setting $M_{lump} + M_{rod} = M_t$ then

$$M_t = \begin{bmatrix} 12 & -1 \\ -1 & 12 \end{bmatrix}$$

Furthermore, $M_t^{-1} = \frac{1}{143} \begin{bmatrix} 12 & -1 \\ -1 & 12 \end{bmatrix}$

Our state vector is $x = [x_1, x_2, \dot{x}_1, \dot{x}_2]^T$ where

- $z_1 = x_1$
- $z_2 = x_2$
- $z_3 = \dot{x}_1$
- $z_4 = \dot{x}_2$

We then get that $[\dot{z}_1, \dot{z}_2, \dot{z}_3, \dot{z}_4]^T = [z_3, z_4, \ddot{x}_1, \ddot{x}_2]^T$. Write now $\dot{x} = Ax + BF$. Then

$$\begin{bmatrix} \ddot{x}_1 \\ \ddot{x}_2 \end{bmatrix} = M_t^{-1} \left[-C_{rod} \begin{bmatrix} z_3 \\ z_4 \end{bmatrix} - K_{rod} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} + \begin{bmatrix} F \\ 0 \end{bmatrix} \right]$$

I have to break this expression up. I first simplify $-C_{rod} \begin{bmatrix} z_3 \\ z_4 \end{bmatrix}$

$$\begin{aligned} -C_{rod} \begin{bmatrix} z_3 \\ z_4 \end{bmatrix} &= -0.1 \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} z_3 \\ z_4 \end{bmatrix} \\ &= -0.1 \begin{bmatrix} z_3 - z_4 \\ z_4 - z_3 \end{bmatrix} \end{aligned}$$

Now for $-K_{rod} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}$

$$\begin{aligned} -K_{rod} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} &= \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \\ &= \begin{bmatrix} z_1 - z_2 \\ z_2 - z_1 \end{bmatrix} \end{aligned}$$

And finally, $B_0 F = \begin{bmatrix} F \\ 0 \end{bmatrix}$

We now get inside the bracket equation

$$-C_{rod} - K_{rod} + B_0 F$$

which becomes

$$-0.1 \begin{bmatrix} z_3 - z_4 \\ z_4 - z_3 \end{bmatrix} - \begin{bmatrix} z_1 - z_2 \\ z_2 - z_1 \end{bmatrix} + \begin{bmatrix} F \\ 0 \end{bmatrix} = \begin{bmatrix} 0.1z_4 - 0.1z_3 + z_2 - z_1 + F \\ 0.1z_3 - 0.1z_4 + z_1 - z_2 \end{bmatrix} = \psi$$

Now we can solve M_t^{-1}

$$\begin{aligned} M_t^{-1} \psi &= \frac{1}{143} \begin{bmatrix} 12 & -1 \\ -1 & 12 \end{bmatrix} \begin{bmatrix} 0.1z_4 - 0.1z_3 + z_2 - z_1 + F \\ 0.1z_3 - 0.1z_4 + z_1 - z_2 \end{bmatrix} \\ &= \frac{1}{143} \begin{bmatrix} 12(0.1z_4 - 0.1z_3 + z_2 - z_1 + F) - (0.1z_3 - 0.1z_4 + z_1 - z_2) \\ -(0.1z_4 - 0.1z_3 + z_2 - z_1 + F) + 12(0.1z_3 - 0.1z_4 + z_1 - z_2) \end{bmatrix} \\ &= \frac{1}{143} \begin{bmatrix} -12z_1 + 12z_2 - 1.2z_3 + 1.2z_4 - z_1 + z_2 - 0.1z_3 + 0 - 1z_4 + 12F \\ z_1 - z_2 + 0.1z_3 - 0.1z_4 + 12z_1 - 12z_2 + 1.2z_3 - 1.2z_4 - F \end{bmatrix} \\ &= \frac{1}{143} \begin{bmatrix} -13z_1 + 13z_2 - 1.3z_3 + 1.3z_4 + 12F \\ 13z_1 - 13z_2 + 1.3z_3 - 1.3z_4 - F \end{bmatrix} \end{aligned}$$

Since $\dot{z}_3 = \ddot{x}_1$ and $\dot{z}_4 = \ddot{x}_2$, we can find that

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ \frac{-13}{143} & \frac{13}{143} & \frac{-1.3}{143} & \frac{1.3}{143} \\ \frac{13}{143} & \frac{-13}{143} & \frac{1.3}{143} & \frac{-1.3}{143} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ \frac{12}{143} \\ \frac{-1}{143} \end{bmatrix}$$

$$C = [0 \quad 1 \quad 0 \quad 0]$$

$$D = 0$$

Since we have that $y = x_2 = z_2$, and our state vector is $z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \dot{x}_1 \\ \dot{x}_2 \end{bmatrix}$ we can then derive the

C matrix as

$$y = [0 \quad 1 \quad 0 \quad 0] \begin{bmatrix} x_1 \\ x_2 \\ \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = x_2$$

- (b) Use Matlab to find the transfer function of the system, and verify the locations of the poles and zeros

Ans: We can find the transfer function and verify that the poles are the exact same as the eigenvalues for A

```
A = [0 0 1 0;
      0 0 0 1;
      -13/143 13/143 -1.3/143 1.3/143;
      13/143 -13/143 1.3/143 -1.3/143];
B = [0;0;12;-1];
C = [0 1 0 0];
D = 0;
```

```
O = obsv(A, C);
```

```
sys = ss(A, B, C, D);
poles = eig(A);
zeros = tzero(sys);
```

```
sys_tf = tf(sys);
[num, den] = tfdata(sys_tf, 'v');
roots_den = roots(den);
```

```
disp(roots_den)
disp(poles)
```

The output of the code is the following

```
-0.0091 + 0.4263i
-0.0091 - 0.4263i
-0.0000 + 0.0000i
-0.0000 + 0.0000i

-0.0091 + 0.4263i
-0.0091 - 0.4263i
-0.0000 + 0.0000i
-0.0000 + 0.0000i
```

We have that the transfer function is

$$\frac{-0.006993s^2 + 0.000699s + 0.006993}{s^4 + 0.01818s^3 + 0.1818s^2}$$

This is found with the following matlab commands

```
A = [0 0 1 0;
0 0 0 1;
-13/143 13/143 -1.3/143 1.3/143;
13/143 -13/143 1.3/143 -1.3/143];
B = [0; 0; 12/143; -1/143];
C = [0 1 0 0];
D = 0;

sys = ss(A, B, C, D);
[num, den] = tfdata(sys, 'v');
num = round(num, 6); den = round(den, 6);
disp('Clean transfer function:');
sys_tf_clean = tf(num, den);
```

- (c) The desired output acceleration of the system is $\ddot{y} = A \sin(\omega t)$. Choose A such that the output reaches 1 after one period $T = 10s$, and set it so that the acceleration reaches zero after T . Finally, obtain and plot the desired velocity and position as a function of time from $t_0 = 0$ to $t_f = T$.

Ans: We solve first for the angular frequency ω .

$$\omega = \frac{2\pi}{T} = \frac{2\pi}{10} = 0.2\pi$$

So the angular frequency is 0.2π radians per second second

We can find velocity by integrating the acceleration $\ddot{y} = A \sin(\omega t)$

$$\begin{aligned}\dot{y} &= \int A \sin(\omega t) dt \\ &= -\frac{A}{\omega} \cos(\omega t) + C_1\end{aligned}$$

And further for getting the position

$$\begin{aligned}y &= -\frac{a}{\omega} \int \cos(\omega t) + C_1 dt \\ &= \frac{-A}{\omega^2} \sin(\omega t) + C_1 t + C_2\end{aligned}$$

Now we can apply the boundary conditions. The output y should reach 1 at the end of the period. And the acceleration should be 0 at the end of the period

- $y(T) = 1$
- $\ddot{y}(T) = 0$

We found first that the angular velocity is $\omega = 0.2\pi$. Put this into $\ddot{y}(T) = A \sin(\omega T)$ where $T = 10$ and we see this is satisfactory for any given A !

If we are now to solve for the constants, we need initial conditions. Lets say that the speed and position is 0 at $t = 0$

$$\begin{aligned}\dot{y}(0) &= -\frac{A}{\omega} \cos(0) + C_1 \\ &= 0 \\ -\frac{A}{\omega} &= C_1\end{aligned}$$

From $y(0) = 0$, we get that

$$\begin{aligned}y(0) &= -\frac{A}{\omega^2} \sin(0) + 0 \cdot C_1 + C_2 \\ &= 0C_2 &= 0\end{aligned}$$

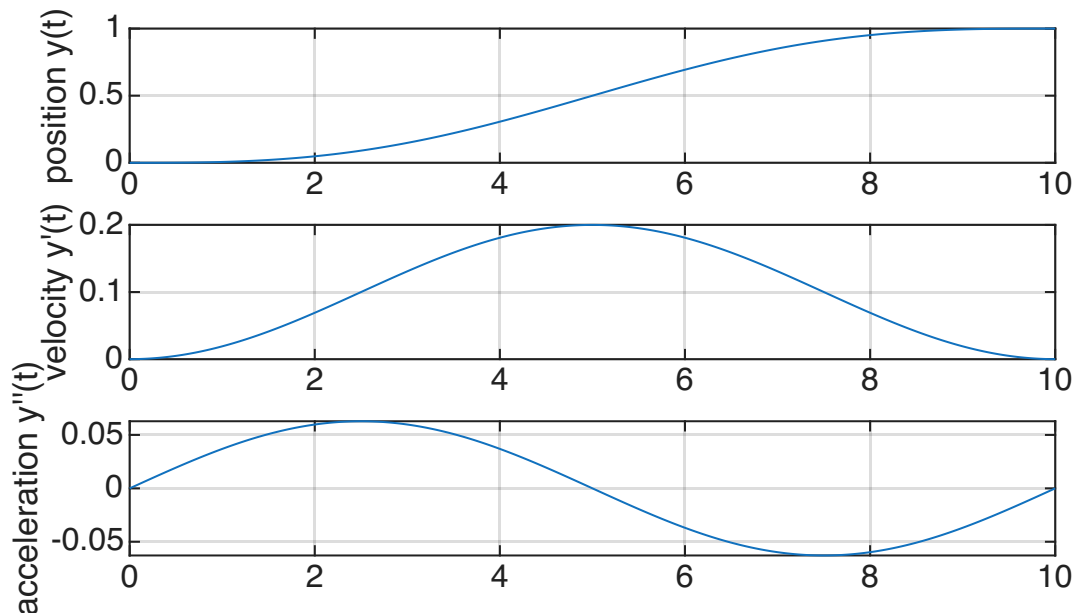
And we get our final position function to be

$$y = -\frac{A}{\omega^2} \sin(\omega t) + \frac{A}{\omega} t$$

And now all remains is to solve for the final A that gives the final position value to be 1

$$\begin{aligned}y(T) &= -\frac{A}{\omega^2} \sin(\omega T) + \frac{A}{\omega} T \\ &= \frac{A}{\omega} T = 1 \\ A &= \frac{\omega}{T} \\ &= 0.02\pi\end{aligned}$$

Here is the plot that shows the trajectories of $\ddot{y}$, $\dot{y}$, y



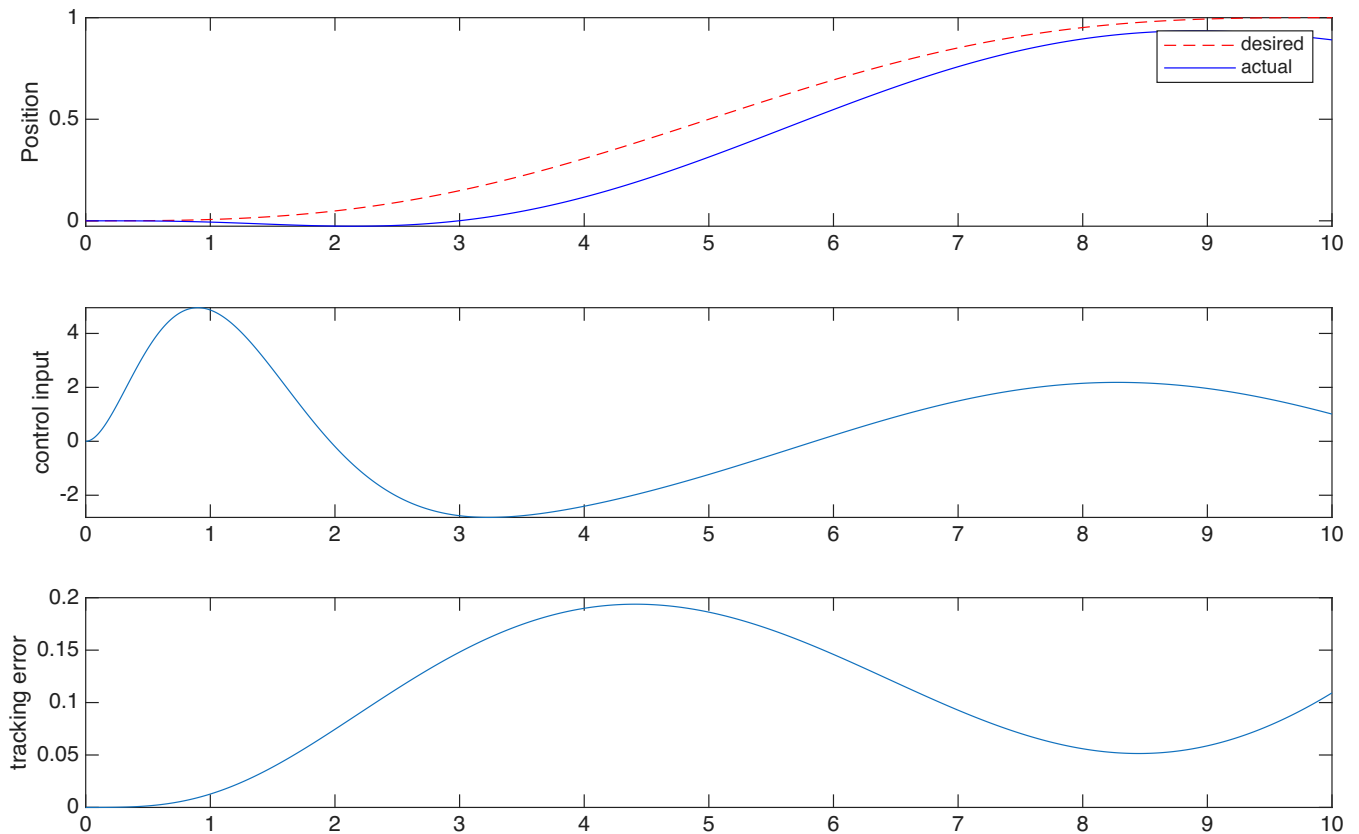
- (d) Choose your favourite feedback system to track the desired reference output (for example, LQR) and explain your design decisions.

Ans: I first want to mention that our system is nonminimum phase. Because when we initially push mass 1, the distance between mass 1 and mass 2 will initially decrease, and then start to increase above the threshold it started off at t_0 .

For my design decisions, I use a lqr controller with the states $x = [0, x_2, 0, \dot{x}_2]$.

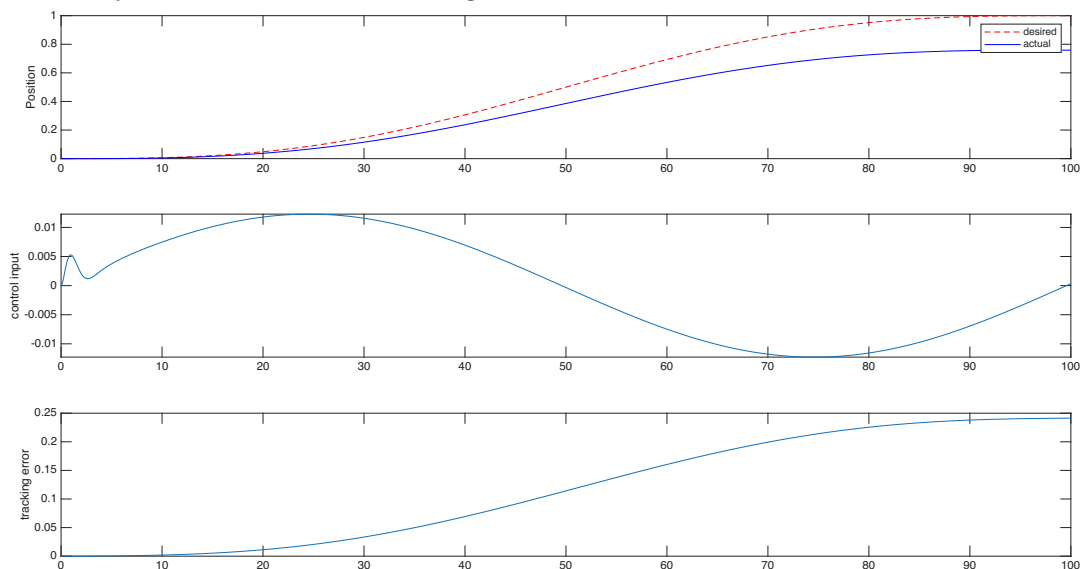
- (e) Simulate and plot the output tracking of the system with this feedback (for example, using LSIM in Matlab). What happens to the tracking performance if T is varied?

Ans: From the script that's in the code appendix, I get the following plot of the system, where I have included the desired trajectory we calculated in 1c), and the actual trajectory we get from the lqr controller



From this, we see that the initial position first decreases, and then starts to follow the desired trajectory with a delay. This is due to the non minimum phase property of our system. If we increase T , we see that the state doesn't converge to our desired trajectory.

As we vary T , we don't see any improvements. The tracking errors are linearly proportional to the time, and will only see an increase as time goes on.



The y_d trajectory continues to end at 1, because we just vary the angular velocity ω

(f) Find the internal dynamics of the system, then find the reference state tra-

jectory x_{ref} and the inverse input u_{inv} . Plot them, and describe how you found them.

Ans: We can first state that from devasia on feedforward control, then

$$\begin{aligned}\frac{d^r}{dt^r}y(t) &= CA^r x(t) + CA^{r-1}Bu(t) \\ &= A_y x(t) + B_y u(t)\end{aligned}$$

where r is the difference in degrees of our transfer functions numerator and denominator. From 1b), we can obviously see that $r = 2$ in our case ($r = \text{difference in degree of numerator and denominator of the transfer function}$).

We want to find a transformation matrix T such that

$$z = Tx = \begin{bmatrix} \zeta \\ \eta \end{bmatrix} = \begin{bmatrix} y \\ \dot{y} \\ \eta_1 \\ \eta_2 \end{bmatrix}$$

From our system, we know that

$$y = x_2 = Cx = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} x$$

and

$$\dot{y} = CAx = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} A = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

Thus we know that $T_\zeta = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. We are going to find $Tx_{ref} = \begin{bmatrix} \zeta_d(t) \\ \eta_{ref}(t) \end{bmatrix}$ where we calculate η_{ref} from our internal dynamics. Thus, we need to find a T_η that makes

$$T = \begin{bmatrix} T_\zeta \\ T_{eta} \end{bmatrix}$$

invertible. An easy solution to this is to pick

$$T_{eta} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Then T basically has the same form as a permutation matrix, which automatically has the inversion property

The internal dynamics are given as

$$\dot{\eta} = A_{inv}\eta + B_{inv}\mathcal{Y}_d$$

where

- $A_{inv} = T_\eta [A - BB_y^{-1}A_y] T_r^{-1}$
- $B_{inv} = [T_\eta [A - BB_y^{-1}A_y] T_l^{-1} : T_\eta BB_y^{-1}]$

- $\mathcal{Y}_d = [\zeta_d \ \ddot{y}_d]^T = [y_d \ \dot{y}_d \ \ddot{y}_d]$

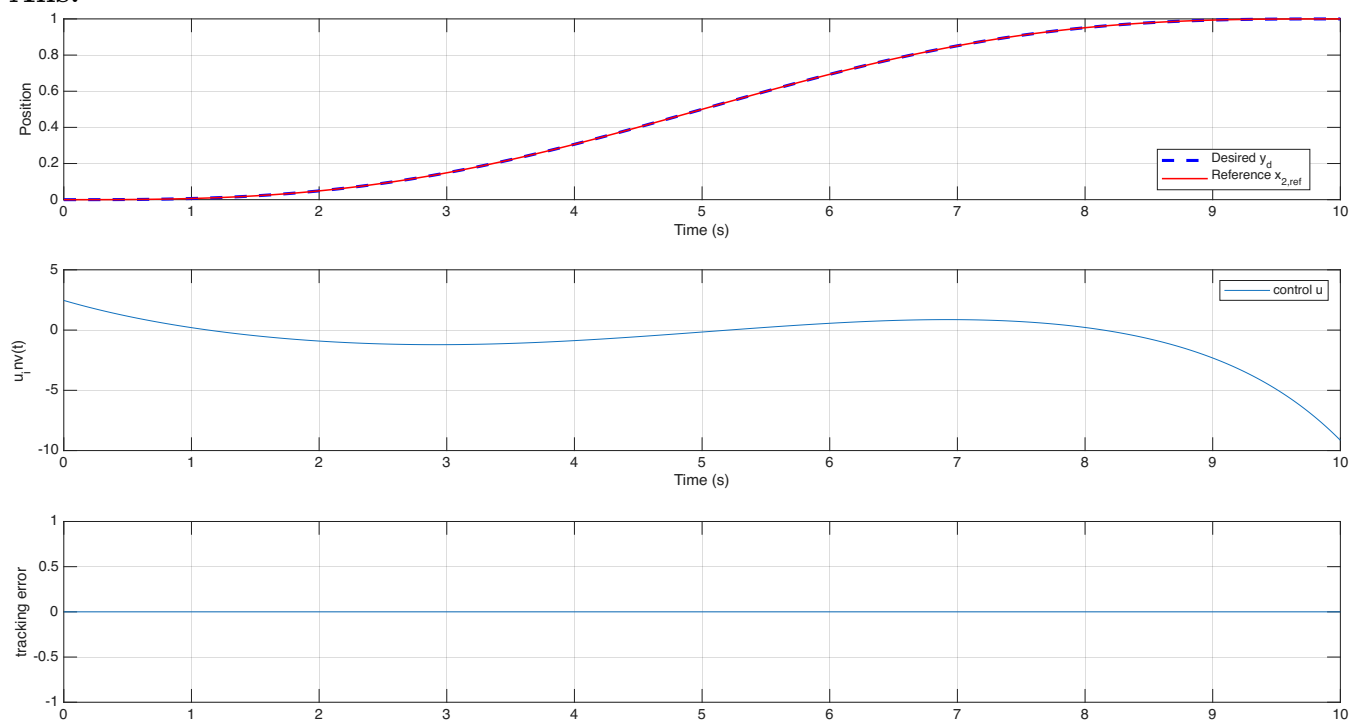
Since our system is non minimum phase, the 2×2 matrix A_{inv} has a positive unstable eigenvalue. We need to decouple A. This allows us to solve for the stable part causally forward in time, and the unstable part backwards noncausal. We know that A is diagonalizable, we write $A_{inv} = VDV^{-1}$. So in this new decoupled system, we get the following internal dynamics

$$\frac{d}{dt} \begin{bmatrix} \eta_s \\ \eta_l \end{bmatrix} = \begin{bmatrix} A_s & 0 \\ 0 & A_l \end{bmatrix} \begin{bmatrix} \eta_s \\ \eta_l \end{bmatrix} + \begin{bmatrix} B_s \\ B_l \end{bmatrix} \mathcal{Y}$$

where A_s is the part that contains the stable eigenvalues, and A_l contains the unstable parts.

(g) Simulate and plot the system with this feedforward input.

Ans:



As you can see, we have a **big** improvement from our feedforward controller. This can confirm that feedforward control largely improves performance of non-minimum phase systems, to the point where we have perfect control. We also start off with initial non-zero control effort, due to the anticipation. Amazing

(h) Add the feedforward and feedback inputs together, and simulate the results in Matlab. Which setup has the best tracking performance (i.e., feedback, feedforward, feedback + feedforward)?

Ans:

I measure the best model with the following metric

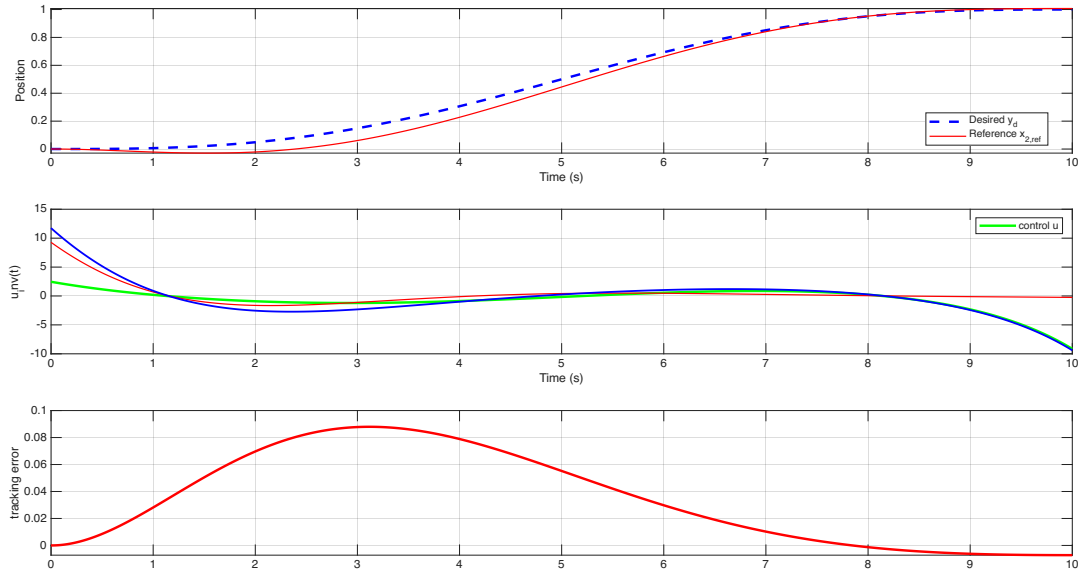
```
tracking_error = y_desired - y_simulated
error_metric = trapz(tracking_error)
```

I will show a table with the metrics measured after discussing the results.

First, I display the solution for the added control of the feedback and feedforward

$$u_{total} = u_{fb} + u_{ff}$$

This gives the following graph.



Very nice. Since I have already showed the plots with the controls gathered from feedback and feedforward from 1d) and 1f), I will just refer you back there.

We see that the worst performing controller, is the one purely using feedback. This is because it will only regulate itself as it goes. But since our system has non-minimum phase, we need to be able to predict future behaviour and adjust our controls according to the predictions we have. This allows us to anticipate future behaviour of our system.

The table below has all the same environment assumption. All have the same time and the same desired output. The metrics take the integral of the difference between desired and actual trajectory. So the bigger the number, the more worse control

Control Method	Tracking Error	Control Effort
FB	101.3295	509.1809
FF	0	-635.3090
FB + FF	34.8750	-392.8959

Table 1: Performance Comparison of Control Methods

To conclude, the feedforward + feedback control is the most chill with a an impressive control effort. The purely feedforward control has achieved perfect control, but too high negative control effort, means that there is more effort in the opposite way we are pushing mass 1. So I would say that the feedback + feedforward control is the best, considering it has the smallest magnitude of all control efforts, as well as a reasonably low tracking error. Ultimately, it depends on your priorities.

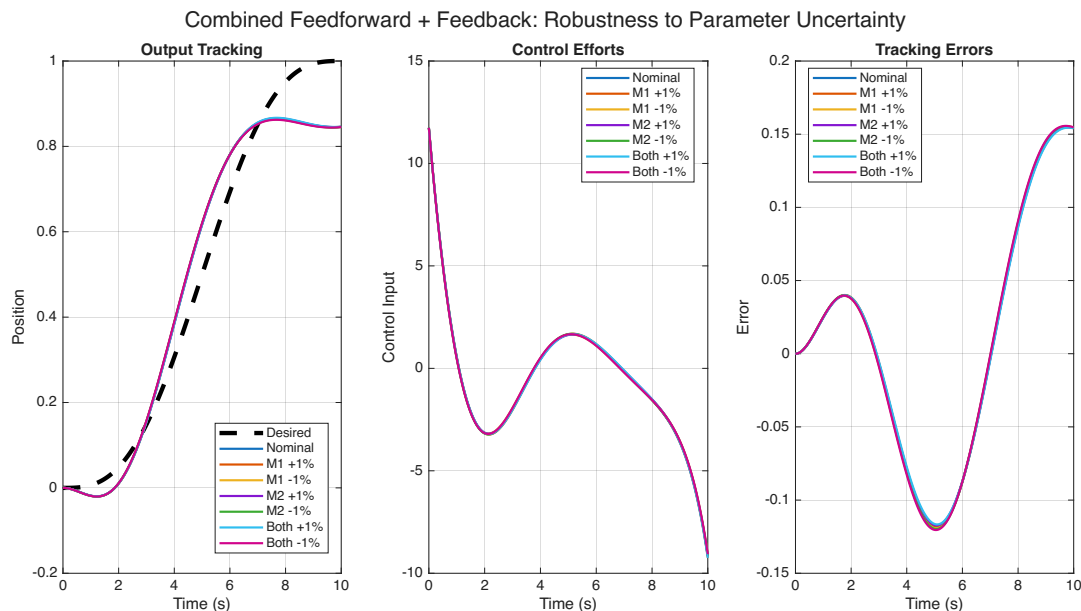
- (i) Using the feedback + feedforward controller in the previous part, modify the system slightly before simulation (for example, modify M_1 and M_2 by

$\pm 1\%$) in order to simulate uncertainty in the model parameters. Which setup has the best tracking performance (i.e., feedback, feedforward, feedback + feedforward)?

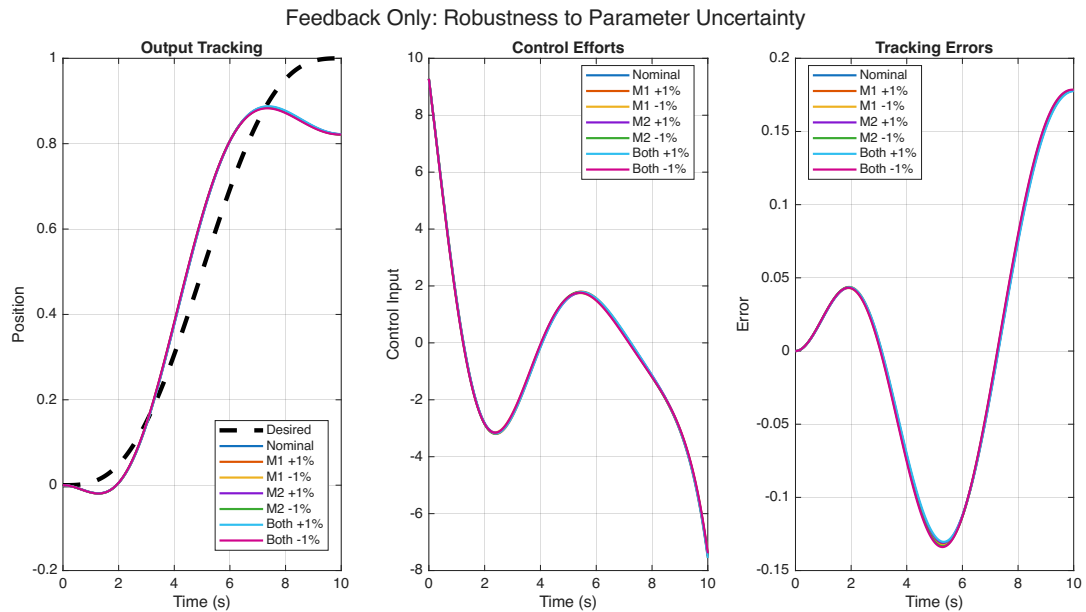
Ans: The combined feedback + feedforward controller demonstrates the best performance of all controllers under uncertainty in the mass we initially push, with negligible performance across all tested cases. The cases I tried was the following.

1. **Nominal:** $M_1 = 10 \cdot 1$, $M_2 = 10 \cdot 1$
2. $M_1 + 1\%$: $M_1 = 10 \cdot 1.01$, $M_2 = 10 \cdot 1$
3. $M_1 - 1\%$: $M_1 = 10 \cdot 0.99$, $M_2 = 10 \cdot 1$
4. $M_2 + 1\%$: $M_1 = 10 \cdot 1$, $M_2 = 10 \cdot 1.01$
5. $M_2 - 1\%$: $M_1 = 10 \cdot 1$, $M_2 = 10 \cdot 0.99$
6. **Both** $+1\%$: $M_1 = 10 \cdot 1.01$, $M_2 = 10 \cdot 1.01$
7. **Both** -1% : $M_1 = 10 \cdot 0.99$, $M_2 = 10 \cdot 0.99$

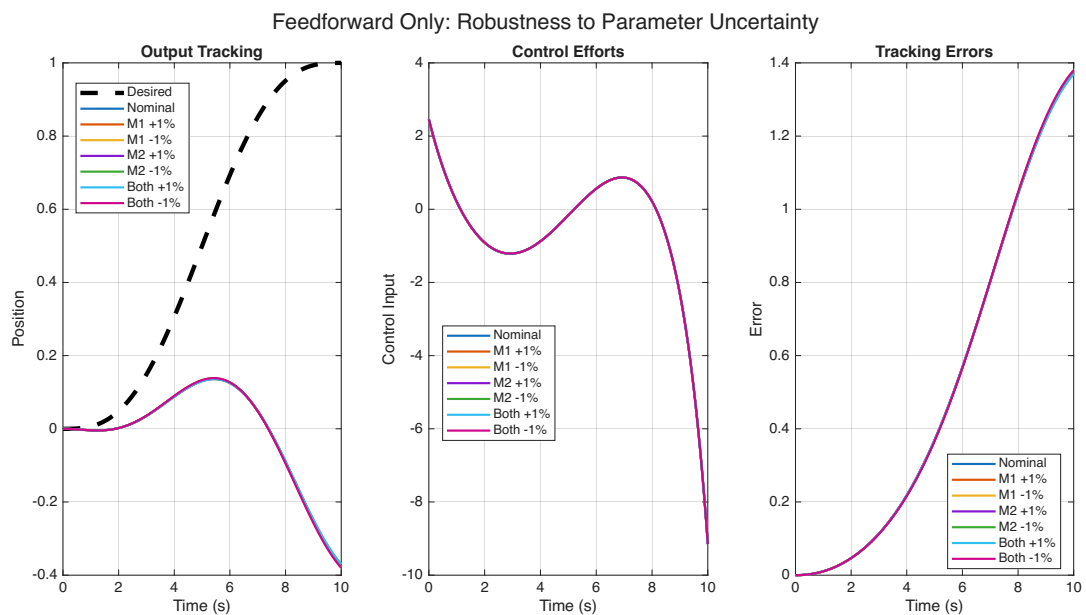
Here is the solution after simulating with feedback + feedforward



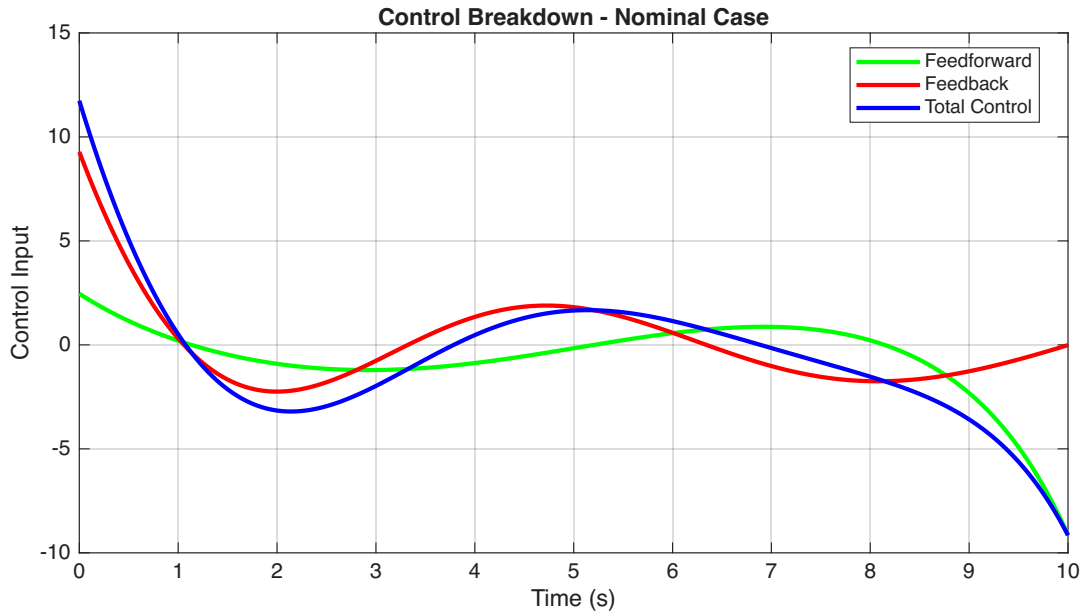
here is the solution after simulating with purely feedback control. As you can see, the performance is still quite good. This makes sense, since the feedback control will regulate itself on the proper path with the gain matrix when we use LQR.



But for Feedforward control, the performance seems to be horrible. This confirms that FF performs bad under uncertainty and noisy systems. This is likely due to the fact that it is solved backwards, and the control is then not consistent with two different simulations



Even though feedback and feedforward + feedback looks awfully similar, you can see from this illustration of control effort between the two different controls, that they differ the most at the end of the time period



2. (20 points) Estimate the resistance value starting from a series of repeated current and voltage measurements. Assume the true relationship

$$u_0(t) = R_0 i_0(t), \quad t = 1, 2, \dots, N,$$

where u_0, i_0 are the exact (noise-free) voltage and current.

Two different voltmeters are used, producing two data sets: one with low noise and one with high noise.

- **Data generation.** Generate an experiment with N measurements where i_0 is uniformly distributed in $[-0.01, 0.01]$ A and $R_0 = 1250 \Omega$. The current is measured without errors, while the voltage measured by the two voltmeters is disturbed by independent, zero-mean, normally distributed noise n_u :

$$n_u \sim \mathcal{N}(0, \sigma_u^2), \quad \sigma_u^2 = \begin{cases} 1, & \text{(good voltmeter),} \\ 16, & \text{(bad voltmeter).} \end{cases}$$

Thus, with the measured signals

$$i(t) = i_0(t), \quad u(t) = u_0(t) + n_u(t), \quad t = 1, 2, \dots, 2N,$$

the first N samples come from the good voltmeter and the next N from the bad one.

- (a) (5 points) **Weighted least-squares (WLS).** Derive the WLS estimate $\hat{R}$ of R by minimizing

$$V_{\text{WLS}} = \frac{1}{2N} \sum_{t=1}^{2N} \frac{(u(t) - R i(t))^2}{w(t)},$$

where $w(t)$ is the variance of the voltage noise corresponding to sample t . Also, derive the **unweighted** least squares estimator, which minimizes,

$$V_{LS} = \frac{1}{2N} \sum_{t=1}^{2N} (u(t) - Ri(t))^2 \quad (5)$$

Ans: To find the optimal $\hat{R}_{WLS}$, we differentiate with respect to R and set the expression equal to 0, to find the given R .

$$\begin{aligned} \frac{\partial V_{WLS}}{\partial R} &= 2 \cdot \frac{1}{2N} \sum_{t=1}^{2N} \frac{i(u - Ri)}{w} \\ &= \frac{1}{N} \sum_{t=1}^{2N} \frac{i u}{w} - R \frac{1}{N} \sum_{t=1}^{2N} \frac{i^2}{w} \\ \frac{1}{N} \sum_{t=1}^{2N} \frac{i u}{w} &= R \frac{1}{N} \sum_{t=1}^{2N} \frac{i^2}{w} \\ \hat{R}_{WLS} &= \frac{\sum_{t=1}^{2N} \frac{i(t)u(t)}{w(t)}}{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}} \end{aligned}$$

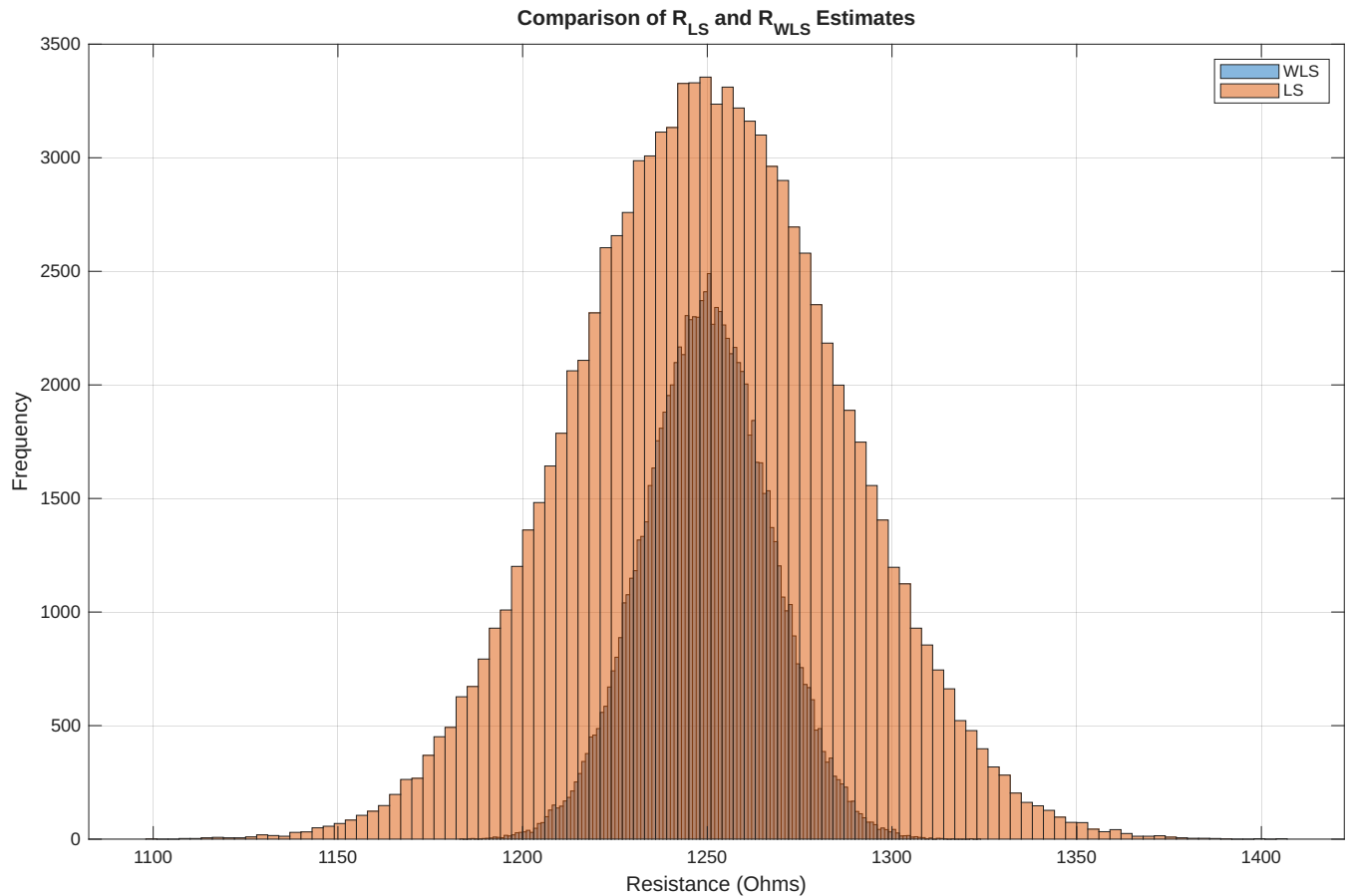
And for the LS method, without the weights added, we get

$$\begin{aligned} \frac{\partial V_{LS}}{\partial R} &= 2 \cdot \frac{1}{2N} \sum_{t=1}^{2N} i(u - Ri) \\ &= \frac{1}{N} \sum_{t=1}^{2N} i u - R \frac{1}{N} \sum_{t=1}^{2N} i^2 \\ \frac{1}{N} \sum_{t=1}^{2N} i u &= R \frac{1}{N} \sum_{t=1}^{2N} i^2 \\ \hat{R}_{LS} &= \frac{\sum_{t=1}^{2N} i u}{\sum_{t=1}^{2N} i^2} \end{aligned}$$

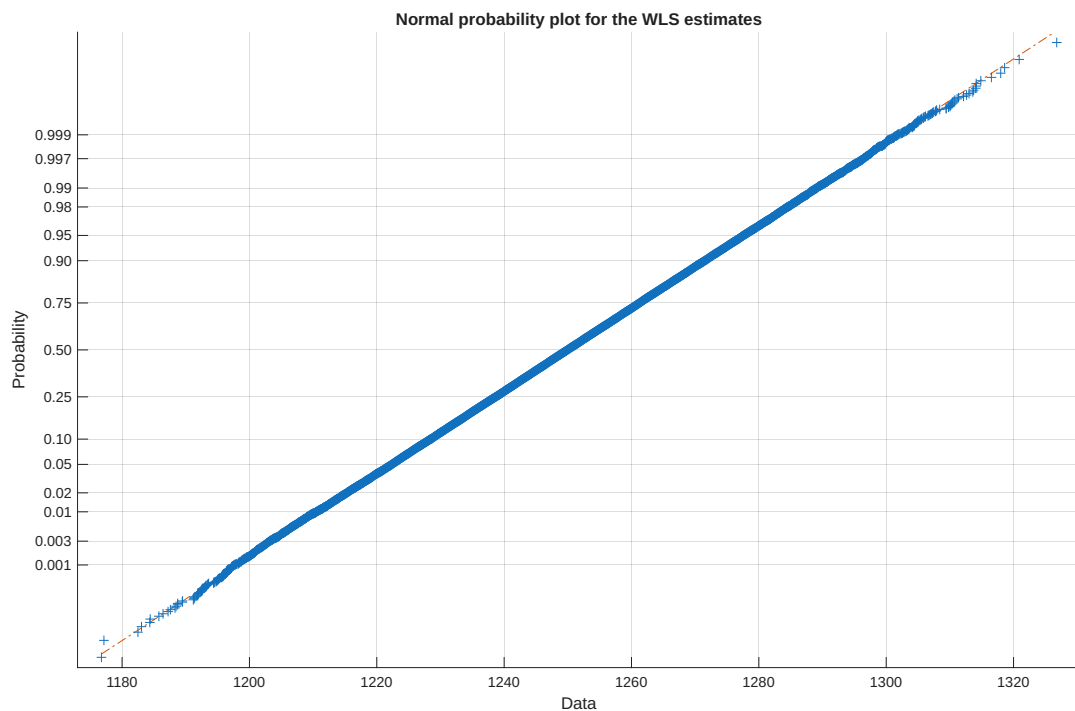
And the question should be answered

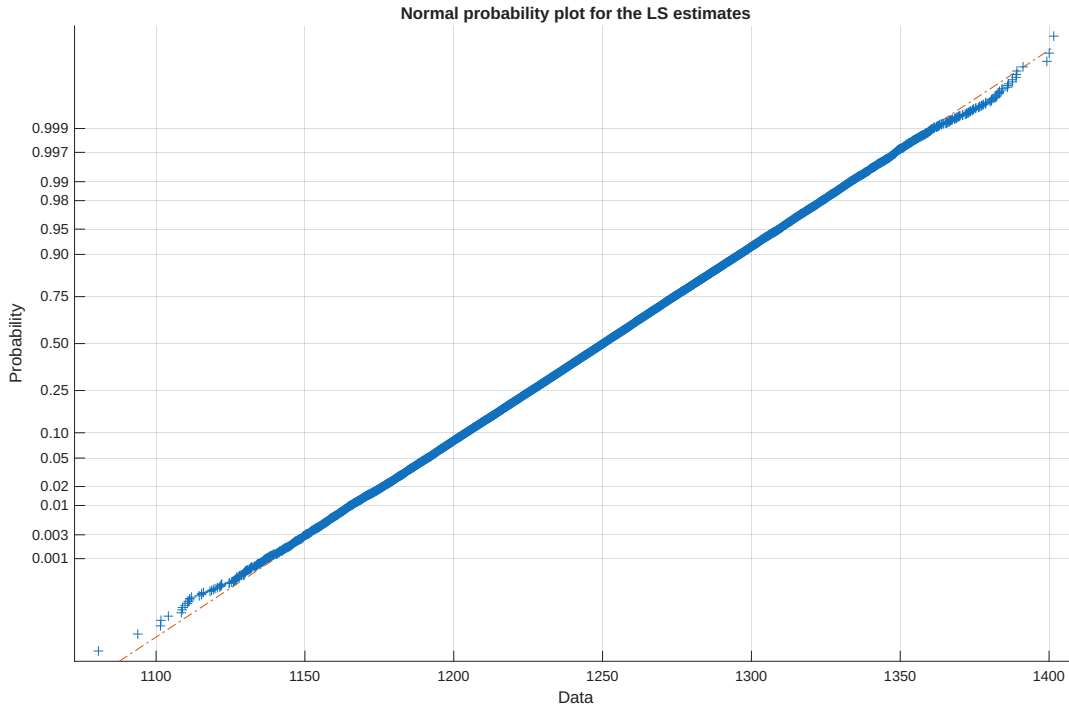
- (b) (5 points) Set $N = 100$. Compute an estimate $\hat{R}$ 10^5 times, using different data each time (according to the Data Generation procedure above), for both the LS and WLS procedures. Plot the resulting values of $\hat{R}$ in two histograms with an appropriate number of bins, one histogram for WLS and one for LS. Discuss the results.

Ans: There should be no surprise that the WLS estimates of the Ohms provide the most accurate ones. This happens because we have two voltmeters, and we give the voltmeter we trust the most, the most weight. This weight is given as the variance of the voltage noise ordered by the voltmeters.



The estimation of $\hat{R}$ over all the 10^5 experiments, leads to the figure above, which looks normally distributed, and indeed it obviously is, from the errors from the 2 voltmeters that are normally distributed





Basically, if the two arrays for the estimates, are closer to the straight line with slope $\frac{1}{1320-1180} = \frac{1}{140}$, the more normal it is distributed. The plots above confirms our belief.

- (c) (10 points) Derive expressions for the variance of the WLS and the LS estimators. Compute the values of each based on the model parameters listed in the Data Generation procedure. Then, compute the sample variance of the estimates $\hat{R}$ in your experiments in part (a). Compare these to the expressions that you derived/computed. What would happen as the number of experiments increases from 10^5 ?

Ans: To derive the variance of the estimators, we take the $\hat{R}_{LS}$ and $\hat{R}_{WLS}$ and set them as inputs to the function $V(x)$

$$\begin{aligned}
 V(\hat{R}_{WLS}) &= V\left(\frac{\sum_{t=1}^{2N} \frac{i(t)u(t)}{w(t)}}{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}}\right) \\
 &= V\left(\frac{\sum_{t=1}^{2N} \frac{i(t)(R_0 i_0(t) + n_u(t))}{w(t)}}{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}}\right) \\
 &= V\left(\frac{\sum_{t=1}^{2N} \frac{i(t)R_0 i_0(t)}{w(t)}}{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}} + \frac{\sum_{t=1}^{2N} \frac{i(t)n_u(t)}{w(t)}}{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}}\right)
 \end{aligned}$$

Since $i(t) = i_0(t)$ is the **true** current, we can substitute that into the first fraction and get

$$V(\hat{R}_{WLS}) = V(R_0) + \frac{\sum_{t=1}^{2N} \frac{i(t)^2 V(n_u(t))}{w(t)^2}}{\left(\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}\right)^2}$$

And since $V(n_u(t)) = w(t)$, we get

$$\begin{aligned} V(\hat{R}_{WLS}) &= \frac{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}}{\sum_{t=1}^{2N} \frac{i(t)^4}{w(t)^2}} \\ &= \frac{1}{\sum_{t=1}^{2N} \frac{i(t)^2}{w(t)}} \end{aligned}$$

Which is the final expression. Its important to note that for a linear equation $a+bx$, then $V(a+bx) = b^2V(x)$ where x is a random variable, which is why the R_0 term disappears

Now for the derivation of $\hat{R}_{LS}$

$$\begin{aligned} V(\hat{R}_{LS}) &= V\left(\frac{\sum_{t=1}^{2N} i(t)u(t)}{\sum_{t=1}^{2N} i(t)^2}\right) \\ &= V\left(\sum_{t=1}^{2N} \frac{i(t)(R_0 i_0(t) + n_u(t))}{i(t)^2}\right) \\ &= V\left(\sum_{t=1}^{2N} \frac{i(t)^2 R_0}{i(t)^2} + \sum_{t=1}^{2N} \frac{i(t)n_u(t)}{i(t)^2}\right) \\ &= V\left(R_0 + \sum_{t=1}^{2N} \frac{i(t)n_u(t)}{i(t)^2}\right) \\ &= \frac{\sum_{t=1}^{2N} i(t)^2 V(n_u(t))}{\left(\sum_{t=1}^{2N} i(t)^2\right)^2} \\ &= \frac{\sum_{t=1}^{2N} i(t)^2 \sigma_u(t)^2}{\left(\sum_{t=1}^{2N} i(t)^2\right)^2} \end{aligned}$$

from the same reasoning that $V(n_u(t)) = w(t)$, and where

$$\sigma_u^2 = \begin{cases} 1, & \text{good voltmeter} \\ 16, & \text{bad voltmeter} \end{cases}$$

Now we can compare the theoretical with the empirical variances. Using the variances derived above, with $N = 100$, and generating $\hat{R}$, $1e5$ times I get:

- **Experimental variance**
 - WLS variance: 300.9280
 - LS variance: $1.9484e + 04$
- **theoretical variance**
 - WLS variance: 265.9457

– LS variance: $1.1347e + 03$

If we increase the number of experiments to $1e6$, we will get more accurate results, where the experimental variance approaches the derived variance we found above. This is the law of large numbers coming in to play.

3. (20 points) **Least-Squares Identification of a One-Parameter Linear System.** Consider the discrete-time system

$$y_k = a y_{k-1} + b u_{k-1} + e_k, \quad k = 2, \dots, N,$$

where

- y_k is the measured output,
- u_k is the known input,
- e_k is a zero-mean white noise sequence,
- a, b are unknown parameters to be estimated.

Stack all samples $k = 2, \dots, N$ to obtain

$$\mathbf{y} = \Phi \theta + \mathbf{e},$$

with

$$\Phi = \begin{bmatrix} y_1 & u_1 \\ y_2 & u_2 \\ \vdots & \vdots \\ y_{N-1} & u_{N-1} \end{bmatrix}, \quad \theta = \begin{bmatrix} a \\ b \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_2 \\ y_3 \\ \vdots \\ y_N \end{bmatrix}, \quad \mathbf{e} = \begin{bmatrix} e_2 \\ e_3 \\ \vdots \\ e_N \end{bmatrix}. \quad (6)$$

- (a) (5 points) By minimizing the sum of squared residuals

$$V(a, b) = \sum_{k=2}^N (y_k - a y_{k-1} - b u_{k-1})^2,$$

show that the optimal parameters satisfy the *normal equations*

$$\Phi^\top \Phi \hat{\theta} = \Phi^\top \mathbf{y},$$

leading to the closed-form least-squares solution

$$\boxed{\hat{\theta} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}.} \quad (7)$$

Ans: To derive the optimal parameters that satisfy the normal equations, we see first that from the exercise, we are given that we have dropped the error terms in the sum. And when we stack all the y_k, u_k , and all a, b that shall be estimated, we get the sum that's given in the exercise. In quadratic form, this turns to

$$\begin{aligned}
r &= y - \Phi\theta \\
V(\theta) &= r^T r \\
&= (y - \Phi\theta)^T (y - \Phi\theta) \\
&= y^T y - y^T \Phi\theta - (\Phi\theta)^T y + (\Phi\theta)^T (\Phi\theta) \\
&= y^T y - 2\theta^T \Phi^T y + \theta^T \Phi^T \Phi \theta
\end{aligned}$$

And now we have to find the optimal parameters a , b that find the minimized sum of squared residuals

$$\begin{aligned}
\frac{\partial V(\theta)}{\partial \theta} &= 0 - 2\Phi^T y + 2\Phi^T \Phi \theta = 0 \\
2(\Phi^T \Phi) \hat{\theta} &= 2\Phi^T y \\
\hat{\theta} &= (\Phi^T \Phi)^{-1} \Phi^T y
\end{aligned}$$

This assumes of course that $\Phi^T \Phi$ is invertible. I will get to this more in 3 c). But all in all, I think the exercise is solved.

- (b) (10 points) Set $N = 100$, $a = 0.2$ and $b = -0.25$, and $e_k \sim \mathcal{N}(0, 0.5)$, and consider the following input sequences:

- a random input that is 1 half the time, and -1 half the time:

$$u_k \sim \begin{cases} 1 & p = 0.5 \\ -1 & p = -0.5 \end{cases} \quad (8)$$

- a random uniform input:

$$u_k \sim \text{randu}[-1, 1] \quad (9)$$

- a gaussian input with mean zero and variance 1:

$$u_k \sim \mathcal{N}(0, 1) \quad (10)$$

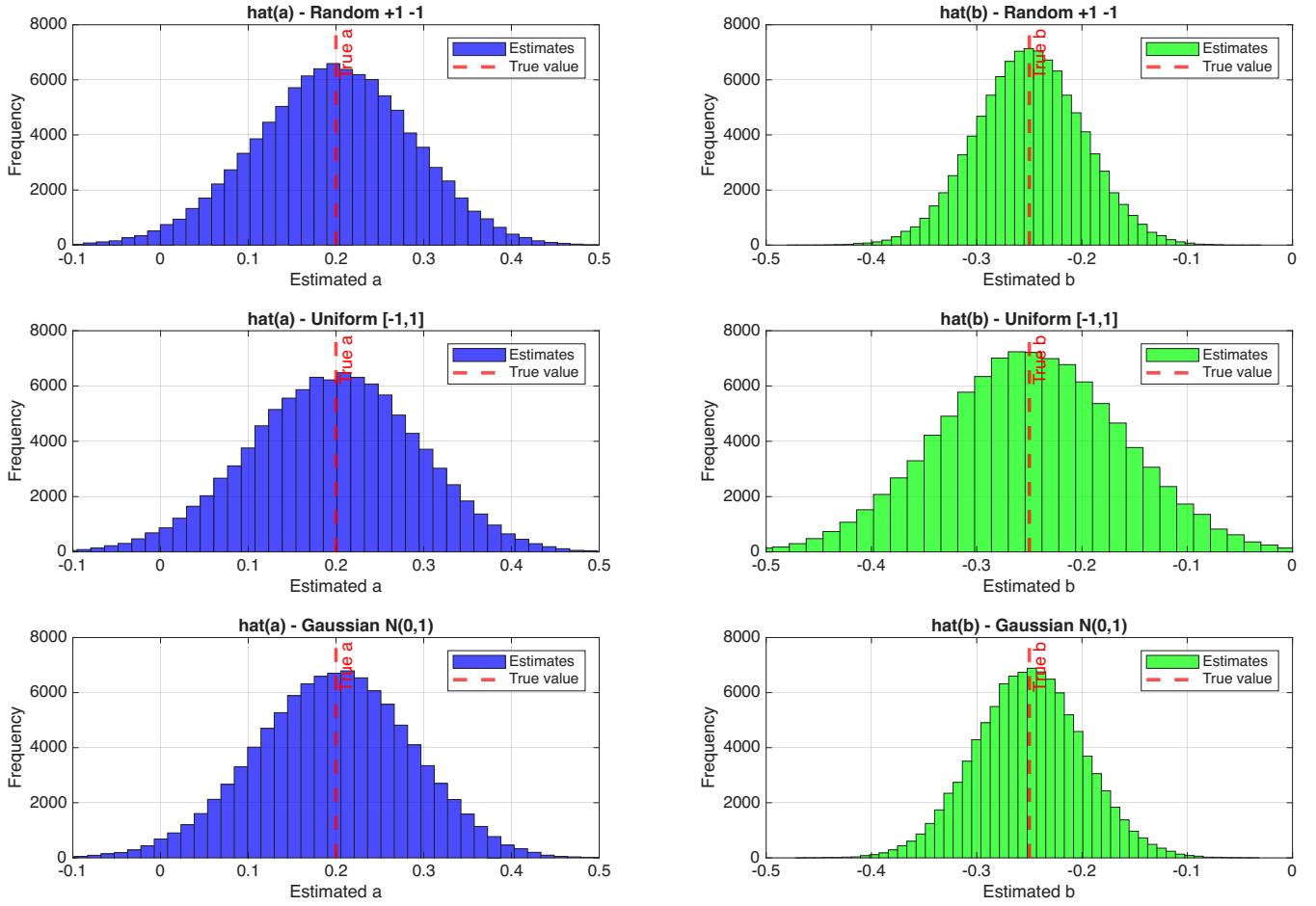
For each input type, perform the following:

- Generate $\mathbf{y}$ using Equation (6)
- Generate $\hat{\theta}$ using Equation (7)
- Repeat this process 10^5 times

Plot histograms for $\hat{a}$ and for $\hat{b}$ for the different input types on the same plot.

Which input type yields the lowest variance estimate?

Ans: I assume that for u_k , that you mean that u is an $n \times 1$ vector, with -1 about half the time, and the other half is 1. Here is the histogram plots for the different a and b :



I will refer to the different control series generation processes like the following:

- **Method 1:** a random input that is 1 half the time, and -1 half the time:

$$u_k \sim \begin{cases} 1 & p = 0.5 \\ -1 & p = -0.5 \end{cases} \quad (11)$$

- **Method 2:** a random uniform input:

$$u_k \sim \text{randu}[-1,1] \quad (12)$$

- **Method 3:** a gaussian input with mean zero and variance 1:

$$u_k \sim \mathcal{N}(0,1) \quad (13)$$

I also gather that the empirical variance of the $\hat{a}$ and $\hat{b}$ arrays vary between the method 1 and method 3. The following table displays the variance of said arrays, where column i is either the $\hat{a}$ array or the $\hat{b}$ array, and the column i is $method_i$

From this, we see that method 2 is without a doubt the worst. It is relatively close between method 1 and method 3 having the lowest variance implication on the coefficient estimates $\hat{a}$ and $\hat{b}$.

It comes to the point where it differs between each time I run the code.

Input Type	Var($\hat{a}$)	Var($\hat{b}$)
Method 1	0.007833	0.002547
Method 2	0.009062	0.007751
Method 3	0.007848	0.002587

(c) (5 points) Discussion

1. Under what conditions is $\Phi^T \Phi$ invertible?

Ans: $\Phi^T \Phi$ is invertible if and only if the following is satisfied; The columns of $\Phi^T \Phi$ have full rank. The only case where this would not happen, was if the controls $u_k = 0 \forall k$. This is unlikely in practice

2. What happens if the input u_k is constant?

Ans: If one of the controls are constant, I dont see any much affection on the series y. The only thing I see happening is that the matrix Φ has two columns that are nearly linearly dependent vectors, which means that they become ill-conditioned, which may result in higher variance. But if $u_k = 0 \forall k$ then the matrix $\Phi^T \Phi$ is non-invertible. I mentioned this earlier. In this case, the estimates $\hat{a}$ and $\hat{b}$ are all zero. This is because the equation

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

would not exist, due non-invertible property of the $\Phi^T \Phi$ matrix

3. How would correlated noise e_k affect the bias of the estimates?

Ans: The linear model introduces bias to the estimates, and it will violate the assumptions of linear regression. This assumption is that e_k is white noise, which is uncorrelated with the data. This will ultimately lead to wrong biased estimates